

APPLIED AI ENGINEERING SERIES

Agentic AI Systems

A Complete Basic-to-Advanced Handbook — Agent Loops, Tools, Memory,
Multi-Agent Architectures, Evaluation & Production Engineering

Agent Loop

ReAct

Tools & MCP

Multi-Agent

Memory

Evaluation

From a single tool-calling loop to orchestrated, self-correcting, production-grade agent systems

Table of Contents

Part A — Agentic AI Fundamentals

- What makes a system 'agentic'
- The agency spectrum: workflows vs agents
- The core agent loop
- When to build an agent (and when not to)

Part B — The Four Core Components

- Component 1: The Model (reasoning engine)
- Component 2: Tools (the action space)
- Component 3: Memory (short & long-term)
- Component 4: Planning & control flow

Part C — Reasoning & Planning Patterns

- ReAct (Reason + Act)
- Chain-of-Thought & Plan-and-Execute
- Reflexion & self-critique
- Tree of Thoughts & search-based planning

Part D — Workflow Architectures

- Prompt chaining
- Routing
- Parallelization (sectioning & voting)
- Orchestrator-Workers
- Evaluator-Optimizer loops

Part E — Multi-Agent Systems

- When multi-agent beats single-agent
- Topologies: supervisor, hierarchical, network, swarm
- Communication & handoff protocols
- Context isolation & the subagent pattern

Part F — Tools, MCP & Environment Design

- Designing tools an agent can actually use
- Model Context Protocol (MCP)
- Code execution as a tool
- Error handling & tool failure recovery

Part G — Memory & Context Engineering

- Context window as a managed resource

Compaction, summarization & note-taking
Vector, episodic & semantic memory
Agentic RAG

Part H — Evaluation, Safety & Production

Evaluating agents (trajectory vs outcome)
Guardrails, permissions & human-in-the-loop
Cost & latency control
Failure modes & debugging playbook

Part I — Reference Implementations & Interview Prep

End-to-end code: ReAct agent from scratch
End-to-end code: supervisor multi-agent system
Framework comparison
Common agentic AI interview questions

PART A

Agentic AI Fundamentals

What Makes a System "Agentic"?

An **agent** is a system where a language model **directs its own process and tool usage in a loop**, deciding what to do next based on what it observes, until it judges the task complete. The defining property is not intelligence — it is **autonomy over control flow**.

The one-sentence test: If a human engineer hardcoded the sequence of steps, it's a **workflow**. If the model decides the sequence at runtime, it's an **agent**. Both are legitimate — and most production "AI agents" are actually workflows, which is usually the right call.

Four properties that distinguish agents from plain LLM calls:

- **Autonomy** — chooses its own next action rather than following a fixed script.
- **Tool use** — can affect the world (search, query a DB, write files, call APIs), not just emit text.
- **Perception/feedback** — observes the result of each action and adapts accordingly.
- **Persistence** — runs over multiple steps toward a goal, maintaining state across them.

The Agency Spectrum

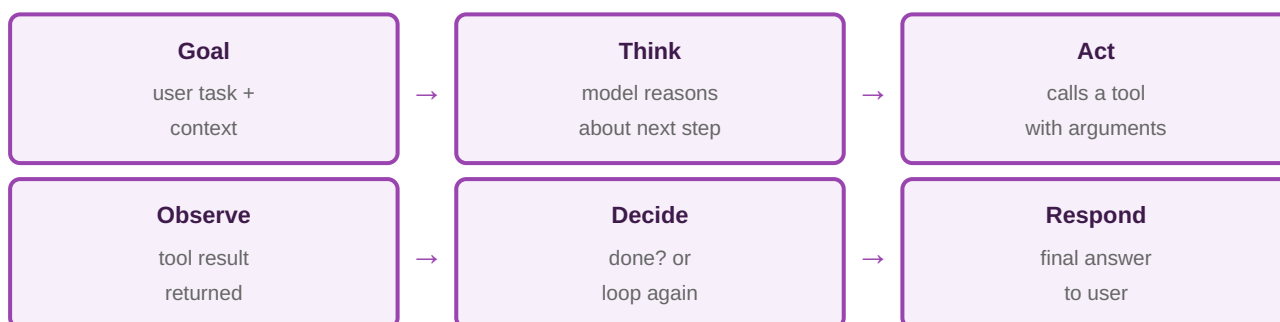
"Agentic" is not binary. It's a spectrum of how much control you hand to the model, and each step up trades reliability and predictability for flexibility.

Level	Pattern	Who Decides Control Flow	Reliability
0	Single LLM call	Developer (nothing to decide)	Highest
1	Chained calls (fixed pipeline)	Developer	Very high
2	Router — model picks a branch	Model picks one of N fixed paths	High
3	Tool-calling loop (ReAct)	Model picks tools & when to stop	Moderate
4	Planning agent (plan then execute)	Model writes its own multi-step plan	Lower
5	Multi-agent system	Model spawns/coordinates other agents	Lowest

Engineering principle: Always use the **lowest level on this ladder that solves your problem**. Each level up multiplies latency, cost, and the surface area for silent failure. Complexity should be earned by evidence that simpler approaches fail, not assumed at design time.

The Core Agent Loop

Strip away every framework and abstraction, and virtually every agent is this loop:



```
# The entire concept of an agent, in ~15 lines.
def agent(goal, tools, max_steps=15):
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                 {"role": "user", "content": goal}]

    for step in range(max_steps):
        response = llm(messages, tools=tool_schemas(tools)) # THINK
        messages.append(response)

        if not response.tool_calls: # model decided it's done
            return response.content # RESPOND

        for call in response.tool_calls: # ACT
            try:
                result = tools[call.name](**call.arguments)
            except Exception as e:
                result = f"ERROR: {e}" # errors are observations too
            messages.append({"role": "tool", "tool_call_id": call.id,
                           "content": str(result)}) # OBSERVE

    return "Stopped: exceeded maximum steps." # always bound the loop
```

Never ship an unbounded loop. Always enforce a hard step limit, a wall-clock timeout, and a token/cost budget. An agent stuck in a retry cycle can burn a large bill in minutes with no output.

When to Build an Agent (and When Not To)

Build an agent when...

Do NOT build an agent when...

The number of steps can't be known in advance	The workflow is the same every time — just write the pipeline
The path depends on intermediate results	A single well-prompted LLM call solves it
The task space is too large to enumerate as branches	Latency budget is tight (agents are multi-call by nature)
Errors are recoverable and verifiable by the agent itself	Mistakes are irreversible or high-stakes without human review
Tool results meaningfully change what should happen next	You can't evaluate whether the agent did the right thing

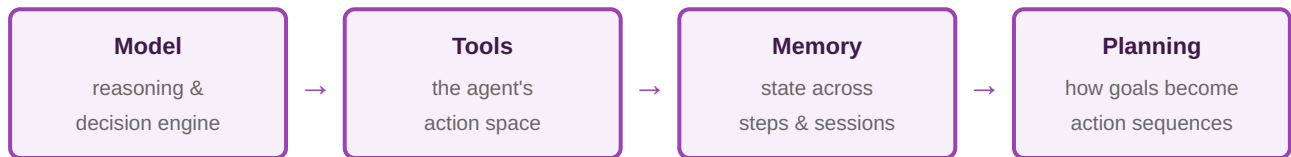
The three questions to ask before building any agent:

1. **Can I verify success programmatically?** If you can't tell whether the agent succeeded, you can't improve it, and you shouldn't automate it.
2. **Are mistakes reversible?** Irreversible actions (sending emails, deleting data, moving money) demand human approval gates.
3. **Is the task genuinely open-ended?** If not, a workflow is cheaper, faster, and far more debuggable.

PART B

The Four Core Components

Every agent — regardless of framework — is assembled from four parts. Weakness in any one caps the performance of the whole system.



Component 1 — The Model (Reasoning Engine)

What matters when choosing a model for agentic use:

- **Tool-calling reliability** — does it emit well-formed arguments consistently? This matters more than raw benchmark scores.
- **Instruction adherence over long contexts** — agents accumulate huge histories; models that drift from the system prompt fail late in a run.
- **Knowing when to stop** — weaker models either loop forever or terminate prematurely. Both are expensive.
- **Reasoning depth** — for planning-heavy tasks, extended reasoning materially improves multi-step decomposition.
- **Cost/latency** — agents make many calls; per-call cost multiplies by trajectory length.

Model routing pattern: Use a strong model for planning and final synthesis, and a cheap fast model for mechanical subtasks (classification, extraction, formatting). This often cuts cost 60–80% with negligible quality loss.

Component 2 — Tools (The Action Space)

Tools define everything the agent *can* do. Agent capability is bounded by tool design far more than by model choice — a brilliant model with badly designed tools performs worse than an average model with excellent ones.

Tool Category	Examples	Design Concern
Retrieval		

	Vector search, SQL query, web search, file read	Return concise, relevant results — not 50KB dumps
Computation	Code execution, calculator, data analysis sandbox	Sandboxing and resource limits
Action / write	Send email, create ticket, commit code, update CRM	Irreversibility — needs approval gates and idempotency
Communication	Ask the user, hand off to another agent, escalate	Clear triggering conditions so it isn't overused
Meta	Plan, reflect, update todo list, mark task complete	Helps structure long trajectories

Component 3 — Memory

Memory Type	Scope	Typical Implementation	Used For
Working / short-term	Current run	The message history in the context window	Immediate reasoning chain
Episodic	Across sessions	Stored summaries of past runs, retrieved by similarity	"Last time this failed because..."
Semantic	Persistent facts	Vector store / knowledge base	Domain knowledge, user preferences
Procedural	How-to knowledge	System prompt, learned playbooks, few-shot examples	Repeatable task recipes
External scratchpad	Current run, off-context	Files the agent writes and re-reads	Beating context limits on long tasks

The file-as-memory pattern: Have the agent write findings to a scratchpad file (e.g., `notes.md` , `todo.md`) and read it back when needed. This survives context compaction, gives the agent durable state, and makes its reasoning inspectable by humans — a remarkably effective and simple technique.

Component 4 — Planning & Control Flow

Three planning strategies:

- **Implicit (reactive)** — the model decides one step at a time with no explicit plan. Simple, adaptive, but can wander on long tasks. This is standard ReAct.
- **Explicit upfront (plan-and-execute)** — generate a full plan first, then execute steps. More token-efficient and auditable, but brittle when reality diverges from the plan.

- **Hybrid (replanning)** — plan upfront, execute, and re-plan when a step fails or new information invalidates assumptions. This is what most robust production agents do.

PART C

Reasoning & Planning Patterns

ReAct — Reason + Act

ReAct is the foundational agent pattern: the model alternates between **Thought** (reasoning about what to do), **Action** (calling a tool), and **Observation** (the tool's result), repeating until done. Interleaving reasoning with acting lets the agent correct course based on real feedback rather than committing to a plan formed in ignorance.

```
# A ReAct trajectory, as it actually appears in the message history:
```

```
User: Which of our top 3 customers by revenue had a support ticket last week?
```

```
Thought: I need revenue data first to identify the top 3 customers.
```

```
Action: query_database("SELECT name, revenue FROM customers ORDER BY revenue DESC LIMIT 3")
```

```
Observation: [{"name": "Acme", "revenue": 480000}, {"name": "Borealis", "revenue": 355000}, {"name": "Cindermill", "revenue": 290000}]
```

```
Thought: Now I need last week's tickets filtered to these three accounts.
```

```
Action: search_tickets(accounts=["Acme", "Borealis", "Cindermill"], since="7d")
```

```
Observation: [{"account": "Borealis", "id": "T-4417", "subject": "API timeout"}]
```

```
Thought: Only Borealis has a ticket. I have everything I need.
```

```
Final Answer: Of your top 3 customers by revenue, only Borealis (#2, $355k) filed a support ticket last week – T-4417, regarding an API timeout.
```

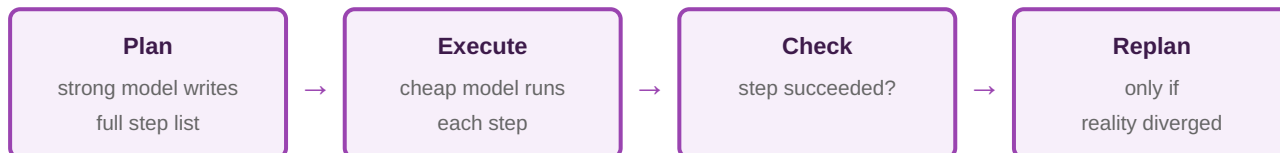
Why ReAct works:

- The explicit Thought step forces the model to justify each action, which measurably reduces spurious tool calls.
- Observations ground the trajectory in real data, sharply reducing hallucination compared to pure chain-of-thought.
- Failures become observations the agent can react to, enabling self-correction without external orchestration.

ReAct failure modes: Looping on the same failing action; over-calling tools for information already in context; and premature termination. Mitigate with step limits, deduplication of identical calls, and an explicit "you already have X — do not re-fetch it" instruction.

Plan-and-Execute

Rather than deciding one step at a time, the agent first produces a complete plan, then executes each step (often with a cheaper model), re-planning only when something breaks.



Aspect	ReAct (reactive)	Plan-and-Execute
Token cost	Higher — full history re-sent every step	Lower — plan once, execute cheaply
Adaptability	High — adjusts every step	Lower — may follow a stale plan
Auditability	Trajectory only visible after the fact	Plan is inspectable/approvable upfront
Best for	Exploratory, unpredictable tasks	Long, structured, multi-step tasks

Reflexion & Self-Critique

Reflexion adds an explicit evaluation step: the agent (or a separate critic) judges its own output, writes a reflection on what went wrong, stores it in memory, and retries with that reflection in context.

- 1 **Attempt** — Agent produces an output or executes a trajectory.
- 2 **Evaluate** — A critic — LLM judge, unit test, linter, or schema validator — scores the attempt.
- 3 **Reflect** — If it failed, the agent writes a concrete explanation of the failure cause.
- 4 **Retry with memory** — The reflection is added to context so the next attempt avoids the same mistake.

Where this shines: Any domain with a **cheap, objective verifier** — code (tests pass/fail), SQL (query executes and returns plausible rows), structured extraction (schema validates), math (checkable). The tighter and more automatic the feedback signal, the more effective the loop.

Where it fails: Subjective tasks with no ground truth. LLM self-critique without an external signal often produces confident but arbitrary revisions — and can make output worse. Prefer real verifiers over self-judgment wherever one exists.

Tree of Thoughts & Search-Based Planning

Instead of one linear chain, the agent explores a **tree** of candidate reasoning paths, evaluates partial states, and backtracks from dead ends — effectively search over reasoning.

- **Generate** multiple candidate next steps at each node instead of committing to one.
- **Evaluate** each partial path (self-scored or via a verifier heuristic).
- **Search** using BFS/DFS/beam search, pruning low-value branches.
- **Cost**: often 10–100× a single chain — justified only for high-value problems where a wrong answer is expensive.

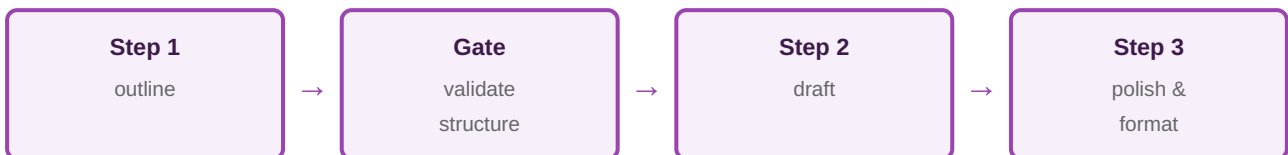
PART D

Workflow Architectures

These five composable patterns cover the overwhelming majority of real production "agentic" systems. Most are workflows, not autonomous agents — deliberately, because they're more reliable.

Pattern 1 — Prompt Chaining

Decompose a task into a fixed sequence of LLM calls, where each step's output feeds the next. Optional programmatic "gates" between steps validate intermediate results.



Use when:

the task decomposes cleanly into fixed subtasks. Each call gets a simpler job, which raises accuracy — trading latency for reliability. **Examples:** generate outline → write sections → edit; extract entities → validate → enrich → store.

Pattern 2 — Routing

A classifier (LLM or model-based) inspects the input and dispatches it to one of several specialized downstream handlers. This is the highest-ROI pattern in most production systems.

```
ROUTES = {
    "billing":    handle_billing,    # specialized prompt + billing tools
    "technical": handle_technical,   # specialized prompt + docs RAG
    "refund":     handle_refund,     # requires approval gate
    "general":    handle_general,    # cheap small model
}

def route(query):
    category = classifier_llm(ROUTING_PROMPT, query) # cheap, fast
    handler = ROUTES.get(category, ROUTES["general"])
    return handler(query)
```

Why routing wins: It lets you optimize each path independently — different prompts, different tools, different models, different guardrails — instead of building one bloated prompt that must handle everything and does each thing poorly. It also enables cost routing: send easy queries to a cheap model, hard ones to a strong one.

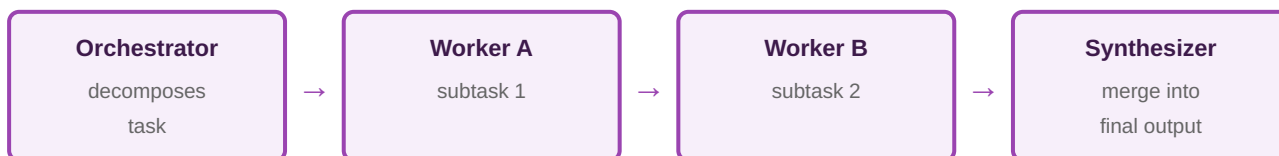
Pattern 3 — Parallelization

Variant	How It Works	Use Case
Sectioning	Split the task into independent subtasks run concurrently, then aggregate	Analyze 20 documents in parallel; run separate guardrail checks alongside the main call
Voting	Run the <i>same</i> task N times (varied prompts/temperature) and aggregate by consensus	Code vulnerability review, content moderation, high-stakes classification

Underused guardrail trick: Run the safety/policy check *in parallel* with the main generation rather than sequentially. You get the safety benefit with near-zero added latency.

Pattern 4 — Orchestrator-Workers

A central orchestrator LLM **dynamically decomposes** a task into subtasks at runtime, delegates each to a worker, and synthesizes the results. Unlike parallelization, the subtasks aren't known in advance.



Key properties:

- The number and nature of subtasks is decided by the model, based on the specific input.
- Workers can run in parallel, each with a **clean context window** — a major advantage for long research tasks.
- **Example:** a research agent that decides to investigate 5 distinct angles, spawns a worker per angle, and writes a synthesized report.

Pattern 5 — Evaluator-Optimizer

One LLM generates, a second evaluates against explicit criteria and returns actionable feedback, and the generator revises. Loop until the evaluator passes it or a retry cap is hit.

```
def evaluator_optimizer(task, max_rounds=3):
    output = generator(task)
    for _ in range(max_rounds):
        verdict = evaluator(task, output)      # {passed: bool, feedback: str}
        if verdict.passed:
            return output
        output = generator(task, revision_feedback=verdict.feedback)
    return output    # return best effort; log that it never passed
```

When this is worth it: Use it when (a) you can articulate clear evaluation criteria, and (b) iteration measurably improves the output — literary translation, complex code, and multi-constraint writing are good fits. If the evaluator can't articulate *why* something failed, the loop just burns tokens.

PART E

Multi-Agent Systems

When Multi-Agent Beats Single-Agent

Legitimate reasons to go multi-agent:

- **Context isolation** — the strongest reason. Each subagent explores with its own clean context window, so a research task spanning far more information than one context window can hold becomes feasible.
- **Parallelism** — genuinely independent subtasks can run concurrently, cutting wall-clock time dramatically.
- **Specialization** — different tools, prompts, and permissions per role, which also narrows each agent's failure surface.
- **Separation of privilege** — the agent that can read production data need not be the one that can write to it.

The honest cost: Multi-agent systems can consume roughly an order of magnitude more tokens than a single chat interaction, and introduce coordination failure modes that don't exist in single agents. Use them when the task value justifies it — heavy parallelizable research, large-scale code migration — not because the architecture diagram looks impressive.

Multi-Agent Topologies

Topology	Structure	Strengths	Weaknesses
Supervisor	One coordinator routes work to specialist agents and collects results	Clear control, easy to debug, predictable	Supervisor is a bottleneck and single point of failure
Hierarchical	Supervisors of supervisors; teams of teams	Scales to very complex decompositions	Deep chains amplify latency and error propagation
Network / peer-to-peer	Agents talk directly to any other agent	Flexible, no bottleneck	Hard to control, can loop indefinitely, expensive
Swarm / handoff	Control is explicitly passed from agent to agent; one active at a time	Simple state model, natural for staged processes	Sequential — no parallelism benefit

Pipeline	Fixed sequence of specialized agents	Most predictable; really a workflow	No adaptability
-----------------	--------------------------------------	-------------------------------------	-----------------

Default recommendation: Start with **supervisor**. It captures most of the benefit of multi-agent (specialization, parallel workers, context isolation) while keeping control flow centralized and debuggable. Graduate to network topologies only when you have concrete evidence you need them.

The Subagent Pattern & Context Isolation

The most practically valuable multi-agent pattern: a lead agent delegates a well-scoped investigation to a subagent, which burns its own context exploring, and returns only a **condensed result**.

- 1 Lead agent receives the task** — Decides what independent lines of investigation exist.
- 2 Spawns subagents with precise briefs** — Each brief must specify: objective, output format, tools to use, and boundaries. Vague briefs are the #1 cause of multi-agent failure.
- 3 Subagents work in isolated contexts** — Each explores deeply, consuming its own context window without polluting the lead's.
- 4 Subagents return condensed findings** — Not raw transcripts — a structured summary of conclusions and evidence.
- 5 Lead synthesizes** — Combines findings into the final deliverable, with the full context budget still available for reasoning.

Delegation brief template — be this specific:

```

OBJECTIVE: Determine whether competitor X has launched a product
            in the EU market in the last 12 months.
SCOPE:      Only official sources, press releases, and regulatory filings.
            Do not use forum posts or speculation.
TOOLS:      web_search, fetch_page
OUTPUT:     A JSON object: {launched: bool, date: str|null,
            product_name: str|null, sources: [url]}
BUDGET:     Max 8 tool calls. If inconclusive, return launched=null
            with what you found.

```

Multi-agent failure modes to design against: duplicated work (two subagents research the same thing), conflicting conclusions with no arbitration rule, infinite delegation (agents spawning agents), and context loss at handoff (the receiving agent lacks information the sender assumed was shared). Explicit briefs and a supervisor that tracks assigned scopes prevent most of these.

Tools, MCP & Environment Design

Designing Tools an Agent Can Actually Use

Treat tool design as an interface design problem — but your user is a model that only sees the name, description, and schema. This is often called **agent-computer interface (ACI)** design, and it deserves as much care as a human-facing API.

Rules for good tool design:

- **Descriptive names and docstrings.** The description is the prompt. State what it does, when to use it, and — critically — **when NOT to use it**.
- **Few, well-chosen tools beat many overlapping ones.** If two tools have ambiguous boundaries, the model will pick wrong. Consolidate or sharply differentiate them.
- **Return concise, structured, model-readable results.** Dumping 50KB of raw JSON wastes context and buries the signal. Pre-filter, truncate, and summarize server-side.
- **Make errors instructive.** `"ERROR: date must be YYYY-MM-DD, got '03/15/24'"` lets the agent self-correct; `"Error 400"` does not.
- **Prefer absolute over relative parameters.** Absolute paths and explicit dates avoid a whole class of agent mistakes.
- **Make destructive tools hard to call accidentally** — require explicit confirmation flags, and separate read tools from write tools.
- **Poka-yoke the interface:** design arguments so the mistake is impossible rather than merely documented.

```
# BAD: vague, overlapping, dumps everything
def search(q: str) -> str:
    """Search."""
    return json.dumps(db.query(q))          # 50KB of raw rows

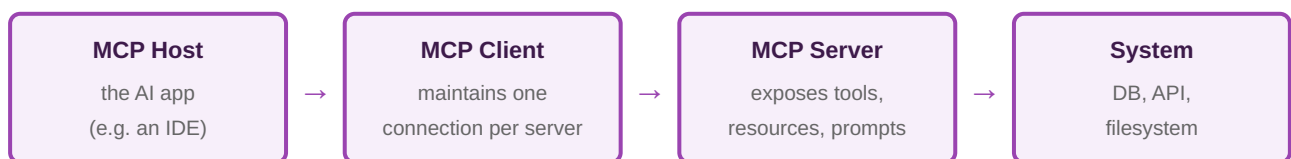
# GOOD: precise contract, bounded output, guidance on usage
def search_customer_tickets(
    account_name: str,
    since_days: int = 30,
    status: Literal["open", "closed", "all"] = "open",
    limit: int = 10,
) -> str:
    """Search support tickets for ONE customer account.

    Use this when you need ticket history for a specific, named account.
    Do NOT use this for aggregate reporting across all customers –
    use `get_ticket_metrics` instead.

    Returns up to `limit` tickets as compact text, newest first.
    Returns "NO_RESULTS" if the account has no matching tickets.
    """
    rows = db.tickets(account_name, since_days, status, limit)
    if not rows:
        return "NO_RESULTS"
    return "\n".join(f"{r.id} | {r.date} | {r.status} | {r.subject}"
                    for r in rows)          # compact, scannable
```

Model Context Protocol (MCP)

MCP is an open standard for connecting AI applications to external tools and data sources — often described as a universal adapter for agent integrations. Instead of writing bespoke glue for every model-to-tool pairing (an M×N problem), each tool implements an MCP server once and any MCP-capable client can use it.



MCP Primitive	What It Provides	Analogy
Tools	Executable functions the model can call (with side effects)	POST endpoints
Resources	Read-only data the app can load into context	GET endpoints / files
Prompts	Reusable templates/workflows the user can invoke	Slash commands

Why it matters architecturally: MCP turns integrations into reusable infrastructure. It also creates a natural permission boundary — you control which servers an agent connects to, and therefore its entire action space, in one place rather than scattered through prompt code.

Security caution: Connecting an agent to an untrusted MCP server is equivalent to granting that server influence over your agent's behavior. Tool descriptions themselves enter the model's context and can carry injected instructions. Vet servers, pin versions, and apply least privilege.

Code Execution as a Tool

One of the most powerful patterns: rather than exposing dozens of narrow tools, give the agent a sandboxed code interpreter and a small set of APIs it can call from within code.

Why code execution is so effective:

- **Composition for free** — the agent can loop, filter, join, and transform without you pre-building a tool for each combination.
- **Massive context savings** — processing 10,000 rows in code and returning a 5-line summary beats loading 10,000 rows into context.
- **Verifiable** — code either runs or throws, giving a crisp, automatic feedback signal for self-correction.
- **Precision** — arithmetic, date handling, and data manipulation are far more reliable in code than in token prediction.

Non-negotiable requirements: run in an isolated sandbox with no host filesystem access, enforce CPU/memory/wall-clock limits, restrict network egress to an allowlist, and never execute agent-written code with production credentials in scope.

Error Handling & Recovery

Failure	Bad Response	Good Response
Malformed tool arguments	Crash the run	Return a validation error naming the bad field and expected format; let the agent retry
Transient API failure	Surface to the agent immediately	Retry with exponential backoff in the tool layer; only escalate after N attempts
Empty results	Return <code>[]</code> or <code>null</code>	Return an explicit <code>"NO_RESULTS"</code> string so the agent doesn't misread it as an error
Agent repeats a failing call	Let it loop	Detect duplicate calls; inject "you already tried this and it failed — try a different approach"
Result too large	Blow the context window	Truncate with a clear marker and offer a pagination or filter parameter

PART G

Memory & Context Engineering

Context as a Managed Resource

Context engineering is the discipline of curating what occupies the model's limited attention at each step. As agents run longer, this becomes the dominant engineering constraint — more so than prompt wording.

Why context degrades over a long agent run:

- **Context rot** — as token count grows, recall of any individual fact declines; attention is finite and spread thinner.
- **Accumulated noise** — failed attempts, verbose tool dumps, and dead ends stay in history and compete for attention.
- **Goal drift** — the original objective, stated 40 messages ago, loses salience against recent content.

Techniques for Managing Long-Running Context

Technique	How It Works	When to Apply
Compaction	Summarize the conversation so far, restart with the summary plus recent messages	When approaching the context limit on a long run
Structured note-taking	Agent writes durable notes to a file, re-reads them after compaction	Long tasks where key findings must survive summarization
Tool result pruning	Drop or truncate old tool outputs that are no longer needed	Tool-heavy trajectories with large outputs
Subagent isolation	Delegate exploration; only the condensed result enters the main context	Research and multi-angle investigation
Just-in-time retrieval	Keep lightweight identifiers (paths, IDs); load full content only when needed	Large corpora or codebases
Goal re-anchoring	Periodically restate the objective and current plan in recent context	Any trajectory over ~20 steps

Compaction done well: When summarizing, explicitly preserve (1) the original goal verbatim, (2) decisions made and their rationale, (3) facts discovered, (4) what has already been tried and failed, and (5) the current next step. Losing item (4) is what causes agents to repeat failed work after compaction.

Agentic RAG

Where classic RAG retrieves once before generating, **agentic RAG** treats retrieval as a tool the agent invokes strategically — deciding whether to search at all, which source to query, and whether the results are sufficient.

Capability	Classic RAG	Agentic RAG
Retrieval decision	Always retrieves	Decides if retrieval is needed at all
Number of retrievals	Exactly one	Iterative — searches again if evidence is thin
Source selection	One fixed index	Routes across multiple indexes, SQL, APIs, web
Query formulation	User query (maybe rewritten)	Agent writes and refines its own queries
Quality control	None	Grades retrieved docs; re-queries if insufficient
Multi-hop	Poor	Native — uses one result to inform the next query

Trade-off: Agentic RAG is materially better on complex, multi-hop, and cross-source questions, but costs several LLM calls per answer and has variable latency. A common production compromise: classic RAG by default, escalate to the agentic path only when the retrieval confidence score is low or the query is detected as multi-part.

Evaluation, Safety & Production

Evaluating Agents

Agent evaluation is harder than model evaluation because there are many valid paths to a correct answer, and a correct answer can result from a badly reasoned trajectory (and vice versa). Measure both.

Dimension	What It Measures	How
Outcome / task success	Did it achieve the goal?	Programmatic checks against a golden result; end-state assertions
Trajectory quality	Was the path sensible?	Tool-call precision/recall vs. a reference trajectory; step count; redundant calls
Tool-use correctness	Right tool, right arguments?	Per-call validation; argument schema conformance rate
Efficiency	Cost per completed task	Tokens, tool calls, wall-clock latency, \$ per success
Safety	Did it avoid disallowed actions?	Adversarial eval set; permission-violation counters
Recovery	Does it handle failure well?	Fault injection — make tools fail and check whether it adapts

Practical evaluation advice:

- **Start with ~20 hand-built cases** covering your real use cases. Teams delay eval waiting for a large dataset; a small set run constantly beats a large set run never.
- **Assert on end state, not exact trajectory** — "the ticket exists with status=resolved" is robust; "called tool X then Y" is brittle.
- **Include adversarial and impossible tasks** — an agent that never says "I can't do that" will fabricate actions.
- **Log full trajectories** — you cannot debug an agent from its final output alone.
- **Fault-inject deliberately** — most production agent failures are unhandled tool errors, not reasoning errors.

Guardrails, Permissions & Human-in-the-Loop

Layered defense for agents:

1. **Least-privilege tools** — the agent should only hold credentials for what it genuinely needs. Separate read and write scopes.
2. **Approval gates on irreversible actions** — sending external communications, spending money, deleting data, deploying code.
3. **Input/output guardrails** — validate inputs; scan outputs for PII, policy violations, and injected instructions.
4. **Sandboxing** — code execution and file access isolated from host and production systems.
5. **Budget limits** — hard caps on steps, tokens, wall-clock time, and spend per run.
6. **Audit logging** — every tool call with arguments, result, timestamp, and actor, retained for review.
7. **Kill switch** — the ability to halt a running agent immediately, and to roll back its actions where possible.

Prompt injection is the defining agent security risk. Any content an agent reads — web pages, emails, documents, tool descriptions, code comments — can contain instructions. Defenses: treat all retrieved content as untrusted data (never instructions), constrain the action space so a hijacked agent still can't do serious damage, require human approval for high-impact actions, and monitor for anomalous tool-call patterns. **Assume injection will succeed sometimes and design the blast radius accordingly.**

Agent Failure Modes — Debugging Playbook

Symptom	Likely Cause	Fix
Loops on the same failing tool call	Uninformative error messages; no loop detection	Make errors instructive; detect duplicate calls and inject guidance
Stops too early with a partial answer	Weak completion criteria in the system prompt	Define explicit done-conditions and require a self-check before finishing
Picks the wrong tool	Overlapping/ambiguous tool descriptions	Consolidate tools; add explicit "do NOT use this for..." guidance
Forgets the goal on long runs	Context rot / goal drift	Re-anchor the objective periodically; compact with goal preserved verbatim
Hallucinates tool results	Tool output format unclear or empty results ambiguous	Return explicit structured markers like NO_RESULTS
Costs explode unpredictably	Unbounded loop; oversized tool outputs	Hard step/token caps; truncate and paginate tool results
Works in testing, fails in production	Real tools fail in ways mocks didn't	Fault-injection testing; retries at the tool layer

Multi-agent duplicates work	Vague delegation briefs; no scope tracking	Explicit briefs with objective/scope/output/ budget; supervisor tracks assignments
--------------------------------	---	---

PART I

Reference Implementations & Interview Prep

Reference 1 — Production ReAct Agent

```

SYSTEM_PROMPT = """You are a data analyst agent.

PROCESS:
1. Think about what information you need before acting.
2. Use tools to gather it. Never guess values you could look up.
3. When you have enough information, give a final answer with figures cited.

RULES:
- Do not re-fetch data already present in this conversation.
- If a tool errors, read the error and adjust – do not repeat the same call.
- If the task cannot be completed with available tools, say so explicitly.
- Stop as soon as the question is fully answered."""

def react_agent(goal, tools, max_steps=15, token_budget=100_000):
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                 {"role": "user", "content": goal}]
    seen_calls, tokens_used = set(), 0

    for step in range(max_steps):
        resp = llm(messages, tools=schemas(tools))
        tokens_used += resp.usage.total_tokens
        messages.append(resp)

        if tokens_used > token_budget:
            # budget guard
            return finalize(messages, reason="budget_exceeded")
        if not resp.tool_calls:
            # natural completion
            return resp.content

        for call in resp.tool_calls:
            sig = (call.name, json.dumps(call.arguments, sort_keys=True))
            if sig in seen_calls:
                # loop detection
                out = ("ERROR: You already made this exact call and it did not "
                      "help. Try a different tool or different arguments.")
            else:
                seen_calls.add(sig)
                try:
                    out = with_retries(tools[call.name], call.arguments)
                    out = truncate(out, max_chars=4000) # protect context
                except ToolError as e:
                    out = f"ERROR: {e.message}. Expected: {e.expected}"

            messages.append({"role": "tool", "tool_call_id": call.id, "content": out})

        if step == max_steps - 3:
            # nudge before hard stop
            messages.append({"role": "user", "content":
                "You are nearly out of steps. Wrap up with your best answer now."})

    return finalize(messages, reason="max_steps")

```

Reference 2 — Supervisor Multi-Agent System

```
SPECIALISTS = {
    "researcher": Agent(tools=[web_search, fetch_page], model="strong"),
    "analyst": Agent(tools=[run_python, query_db], model="strong"),
    "writer": Agent(tools=[], model="fast"),
}

SUPERVISOR_PROMPT = """You coordinate specialist agents.
Available: researcher (finds external info), analyst (computes on internal data),
writer (drafts prose). Decide who acts next, or FINISH if the task is complete.
Give each specialist a brief with: OBJECTIVE, SCOPE, OUTPUT FORMAT, BUDGET.
Never assign the same objective twice."""

def supervisor_system(task, max_turns=10):
    state = {"task": task, "findings": [], "assigned": set()}

    for _ in range(max_turns):
        decision = llm(SUPERVISOR_PROMPT, state, schema=RouteDecision)

        if decision.next == "FINISH":
            break
        if decision.objective in state["assigned"]: # dedupe work
            continue
        state["assigned"].add(decision.objective)

        agent = SPECIALISTS[decision.next]
        result = agent.run(decision.brief) # isolated context
        state["findings"].append({
            "agent": decision.next,
            "objective": decision.objective,
            "result": result, # condensed, not raw
        })

    return SPECIALISTS["writer"].run(
        f"Synthesize a final answer to: {task}\n\nFindings: {state['findings']}")
```

Framework Quick Reference

Framework	Model	Strengths	Consider When
Plain SDK + your own loop	You write the loop	Total control, no abstraction leakage, easiest to debug	Almost always a good starting point
LangGraph	Explicit state graph of nodes/edges	Cycles, checkpointing, human-in-the-loop, durable state	Complex control flow you want to make explicit
LangChain agents	High-level agent abstractions	Fast prototyping, large integration ecosystem	Quick proof-of-concept

LlamaIndex	Data-centric agents	Strong retrieval/indexing primitives	RAG-heavy agents
CrewAI / AutoGen	Role-based multi-agent	Ergonomic multi-agent conversation patterns	Multi-agent experimentation
MCP	Tool/integration protocol (not a framework)	Reusable, portable tool servers	Standardizing integrations across apps

Recommendation: Build your first agent with direct API calls and your own loop. Frameworks hide the loop, and when an agent misbehaves you need to see exactly what went into the context. Adopt a framework once you know precisely which abstraction you want from it.

Common Agentic AI Interview Questions & Model Answers

Q: What's the difference between an AI workflow and an AI agent?

In a workflow, a developer hardcodes the sequence of steps — the LLM fills in content but doesn't choose the path. In an agent, the model decides its own control flow at runtime: which tool to call, in what order, and when it's finished. The distinction matters practically because workflows are far more predictable, cheaper, and easier to debug, so most production systems should be workflows. You escalate to an agent only when the number and order of steps genuinely can't be known in advance.

Q: Walk me through the core agent loop.

It's: think, act, observe, repeat. You seed the context with a system prompt and the user's goal, then call the model with a set of tool schemas. If the model returns tool calls, you execute them, append the results back into the message history as observations, and call the model again. If it returns no tool calls, it has decided it's done and you return its answer. Critically, you always bound the loop with a max step count, a token budget, and a wall-clock timeout, because an agent stuck retrying can burn a large bill with no output.

Q: How do you decide between a single agent and a multi-agent system?

I start with a single agent, because multi-agent adds coordination failure modes and can consume roughly an order of magnitude more tokens. I move to multi-agent for three legitimate reasons: context isolation, where subagents explore with their own clean context windows so the task can span more information than one window holds; genuine parallelism across independent subtasks; and separation of privilege, where the agent that reads production data shouldn't be the one that writes to it. If none of those apply, multi-agent is usually just architecture theater.

Q: What is context engineering and why does it matter for agents?

It's the discipline of curating what occupies the model's limited attention at each step. It matters more for agents than chatbots because agent context grows continuously — every tool result, failed attempt, and reasoning step accumulates. You get context rot, where recall of individual facts degrades as tokens grow,

plus goal drift and accumulated noise. The main techniques are compaction with summarization, structured note-taking to files that survive compaction, pruning stale tool outputs, delegating to subagents so only condensed results return, and periodically re-anchoring the goal.

Q: How would you design tools for an agent?

I treat it as interface design where the user is a model that only sees names, descriptions, and schemas. Concretely: descriptive names and docstrings that say when to use the tool and explicitly when not to; few well-differentiated tools rather than many overlapping ones, since ambiguity causes wrong selections; concise structured returns instead of raw dumps that waste context; instructive error messages that name the bad field and expected format so the agent can self-correct; explicit markers like `NO_RESULTS` so empty results aren't misread as errors; and separating read tools from destructive write tools with confirmation requirements.

Q: What is ReAct and what are its failure modes?

ReAct interleaves Thought, Action, and Observation — the model reasons about what it needs, calls a tool, sees the result, and adapts. Interleaving matters because the agent corrects course on real feedback rather than committing to a plan formed in ignorance. Its failure modes are looping on the same failing action, redundantly re-fetching information already in context, and stopping prematurely. I mitigate with duplicate-call detection that injects guidance to try something different, explicit instructions not to re-fetch, clear completion criteria, and a nudge message a few steps before the hard cap.

Q: How do you evaluate an agent?

On two axes, because a correct answer can come from a terrible trajectory and vice versa. Outcome evaluation asserts on end state — 'the ticket exists with status resolved' — which is robust, unlike asserting an exact tool sequence which is brittle. Trajectory evaluation looks at tool-call correctness, redundant calls, and step count. I'd also measure cost per successful task, safety via an adversarial set, and recovery via deliberate fault injection, since most production agent failures are unhandled tool errors rather than reasoning errors. I'd start with about twenty hand-built cases run constantly rather than waiting for a large dataset.

Q: What is prompt injection in an agentic context, and how do you defend against it?

It's when content the agent reads — a web page, email, document, or even a tool description — contains instructions that hijack its behavior. It's the defining agent security risk because agents both read untrusted content and take real actions. Defenses are layered: treat all retrieved content as untrusted data that is never instructions, and fence it explicitly in the prompt; apply least privilege so a hijacked agent's action space is limited; require human approval for irreversible actions; sandbox code execution; and monitor for anomalous tool-call patterns. The key mindset is assuming injection will sometimes succeed and designing the blast radius accordingly rather than relying on prevention alone.

Q: What's the difference between classic RAG and agentic RAG?

Classic RAG retrieves once, before generating — always, from one index, using the user's query. Agentic RAG treats retrieval as a tool the agent invokes strategically: it decides whether retrieval is needed at all,

which source to query among several, formulates and refines its own queries, grades whether results are sufficient, and searches again if they're thin. That makes it far better at multi-hop and cross-source questions, at the cost of several LLM calls and variable latency. A common compromise is classic RAG by default with escalation to the agentic path when retrieval confidence is low or the query is detected as multi-part.

Q: How do you control cost and latency in an agent system?

Several levers. Model routing — a strong model for planning and synthesis, a cheap fast model for mechanical subtasks like classification and formatting, which often cuts cost sixty to eighty percent. Hard budgets on steps, tokens, and wall-clock time per run. Truncating and paginating tool outputs so large results don't inflate every subsequent call, since agent context is re-sent each step. Loop detection to stop wasted retries. Parallelizing independent subtasks to cut wall-clock time. And architecturally, using the lowest level of agency that solves the problem, because every step up the autonomy ladder multiplies calls.

Final note: The most common mistake in agentic AI is **building an agent where a workflow would do**. Autonomy is a cost, paid in latency, tokens, and debuggability, and it should be purchased only when the task genuinely requires unpredictable control flow. The best agent engineers are distinguished less by the sophistication of their architectures than by their discipline in keeping them as simple as the problem allows — and by their rigor in evaluation, because an agent you cannot measure is an agent you cannot improve.